

El lenguaje C. Conceptos básicos.

Profesores Estructuras de Datos, 2026.

Dpto. Lenguajes y Ciencias de la Computación.

University of Málaga

Licenciado bajo [CC BY-NC 4.0](#)

Aprendiendo a jugar al fútbol



Aprendiendo a programar



```
def getion_greet(name):  
    message == format("Hello, '{name!}")  
    print(message)  
  
if __name__ == '__main__':  
    name = "Alice"  
    greet(name)  
  
__name__ == '__ma  
greet(name)
```

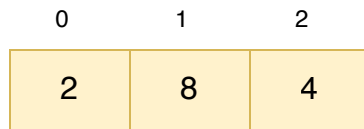
Estructura de Datos (ED)

- Una ED es una forma de organizar los datos que se almacenan en las aplicaciones
- Si definimos una ED eficiente tendremos una aplicación eficiente

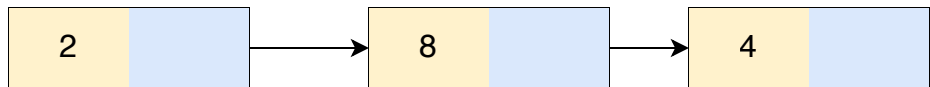
tale.txt (135 635 palabras)	Tiempo	vs. Lista	vs. Array
Lista Enlazada	5.11 segundos		
Array	0.28 segundos	18 veces más rápido	
Árbol	0.14 segundos	37 veces más rápido	2 veces más rápido

Estructura de Datos. Tipos

- Estáticas
- Dinámicas



Estructura de datos estática



Estructura de datos dinámica

Tipo de Datos Abstracto (TDA)

- Un TDA es una especificación independiente de la implementación concreta.
- Describe cómo se deben comportar los datos y las operaciones.
- Ejemplos: listas, pilas, colas, etc.

```
// inserta un par clave-valor en la estructura de datos
```

```
public void put (Key key, Value value);
```

```
// obtiene el valor asociado a una clave o null si la clave no está en la estructura de datos
```

```
public Value get (Key key);
```

```
// verifica si la estructura de datos contiene una clave
```

```
boolean contains(Key key);
```

```
// devuelve todas las claves en la estructura de datos
```

```
public Iterable<Key> keys();
```

Estructuras de Datos en el lenguaje C

Características principales

- Lenguaje de alto nivel no orientado a objetos
- Muy eficiente: características de bajo nivel
- Sistema de tipos débil
- Preprocesador (macros, constantes)
- Acceso directo a memoria (punteros)
- Conjunto reducido de palabras clave
- Tipos de datos estructurados: arrays, estructuras y uniones

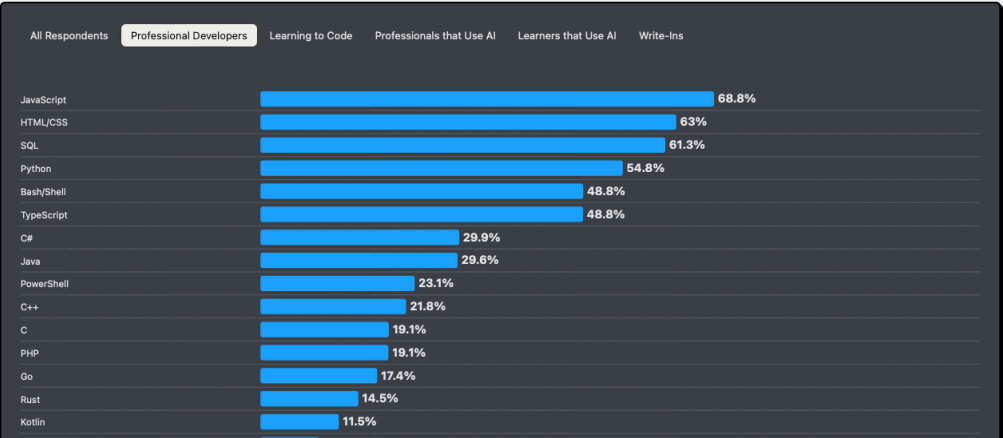
Encuesta sobre programación

Most popular technologies 2.1

Programming, scripting, and markup languages

After more than a decade of steady growth, Python's adoption has accelerated significantly. It saw a 7 percentage point increase from 2024 to 2025; this speaks to its ability to be the go-to language for AI, data science, and back-end development.

Which programming, scripting, and markup languages have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)



Enlace

Qué aporta C

- Control de Bajo Nivel
- Eficiencia y Velocidad
- Ampla Disponibilidad de Bibliotecas
- Portabilidad
- Aprendizaje Fundamental
 - Punteros, estructuras y gestión de memoria

Por qué usar C para definir una Estructura de Datos

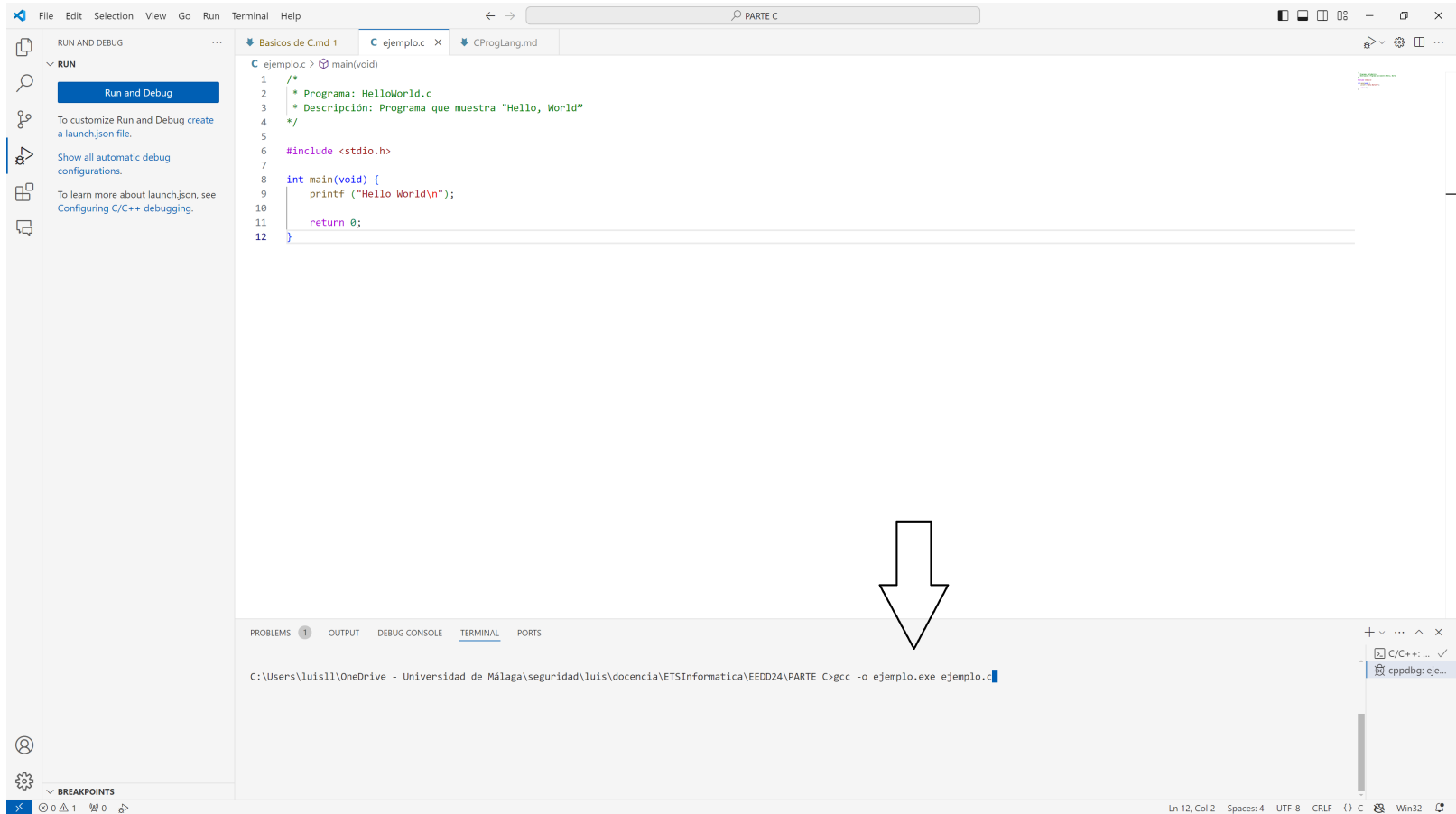
- Control de Memoria
- Flexibilidad en la Implementación
- Punteros y Estructuras



Cómo generar un programa ejecutable

1. Preprocesado
2. Compilación
 - En línea de comandos
 - Integrado en el entorno que se utilice
3. Enlazado (linking)

Primer ejemplo en C. VS Code



Primer ejemplo en C

```
/*
 * Programa: HelloWorld.c
 * Descripción: Programa que muestra "Hello, World"
 * Compilación: gcc HelloWorld.c -Wall -Wextra -Wpedantic -o salida.exe
 * Depuramos y buscamos soluciones.
 */

#include <stdio.h>

int main(void) {

    int a = 42;
    printf("Hi there %i!\n",a);
    return 0;
}
```

Tipos de Datos Simples

Tipo	Descripción	Tamaño (bytes)
char	Byte	1
int	Entero	4
float	Flotante en single precisión	4
double	Flotante en doble precisión	8
short/long int	Entero corto/largo	2/8
unsigned char	Número positivo	1
signed char	Número con signo	1
unsigned int	Entero positivo	4
long double	Flotante con precisión extendida	12

E/S en C

- La mayor parte de las funciones de E/S en C se encuentran en `<stdio.h>`
- La función de salida más utilizada es `printf`

```
int printf(char *format, arg1, arg2, argn)
```

- `printf` convierte, da formato e imprime los argumentos en la salida estándar bajo el control de format.

E/S

Carácter	Impreso como
d,i,l	Número decimal
O	Número octal
X,x	Número hexadecimal
u	Entero sin signo
c	Carácter
s	Cadena de caracteres
f,e,E,g,G	double

E/S Formato `printf`

`%[modificador][ancho][.precisión][longitud][carácter conversión]`

```
#include <stdio.h>

int main() {
    int entero = 42;
    float real = 3.14159;
    char caracter = 'X';
    char cadena[] = "Hola, mundo!";

    // Especificadores de formato:
    printf("Entero: %5d\n", entero);           // Ancho mínimo de 5 caracteres
    printf("Real: %.2f\n", real);             // Dos decimales
    printf("Carácter: %c\n", caracter);
    printf("Cadena: %-20s\n", cadena);        // Justificación a la izquierda, ancho mínimo de 20 caracteres

    return 0;
}
```



E/S

- Implementar un programa que inicialice dos variables enteras y muestre por pantalla la suma de éstas.
- Implementar un programa que inicialice dos variables, una de tipo carácter y otra cadena de caracteres y lo muestre por pantalla.
- Consulta cómo funciona la entrada de datos por teclado, `scanf` e implementa el ejercicio de antes pero leyendo los datos por teclado.

Documentación: [printf](#), [scanf](#)

Sentencias de Control

```
if (CondControl) {  
    accionesSI  
}  
accionseguida
```

```
if (CondControl) {  
    accionesSI  
}  
else {  
    accionesE0C  
}  
accionseguida
```

Sentencias de Control

```
if (CondControl) {  
    accionesSI  
}  
else if (CondControl1) {  
    accionesCC1  
}  
else if (CondControl2) {  
    accionesCC2  
}  
else {  
    accioneselse  
}  
accionseguida
```

```
switch (expresion){  
    case exp-const: acciones  
        break;  
    case exp-const: acciones  
        break;  
    case exp-const: acciones  
        break;  
    default:          acciones  
        break;  
}
```

Sentencias de Control

```
while (expresion) {  
    acciones  
}
```

```
for (expr1;expr2;expr3) {  
    acciones  
}
```

```
do {  
    acciones  
} while
```

Tipo enumerado

```
enum DiaSemana {  
    LUNES,      //0  
    MARTES,     //1  
    MIERCOLES,  //2  
    JUEVES,     //3  
    VIERNES,    //4  
    SABADO,     //5  
    DOMINGO     //6  
};  
...  
enum DiaSemana dia = MARTES;
```

- Cuando se define el tipo enum DiaSemana se crean constantes de entero con signo para cada uno de ellos.

Tipo enumerado

```
enum DiaSemana {  
    LUNES=1,  
    MARTES=2,  
    MIERCOLES=3,  
    JUEVES=4,  
    VIERNES=5,  
    SABADO=6,  
    DOMINGO=7  
};  
...  
enum DiaSemana dia = MARTES;
```

- Cuando se define el tipo enum DiaSemana se crean constantes de entero con signo para cada uno de ellos.



Enumerados

- Crea un enumerado con los meses del año. Haz una aplicación que lea un número entero entre 1 y 12 y muestre el mes del año correspondiente. Cuando el usuario introduce 0, se sale finalizando el programa.

Tipos de Datos Estructurados. Arrays

- Unidimensionales

```
Tipo nombre [tam];
```

Ejemplo:

```
double saldo[10];
```

- Varias dimensiones

```
Tipo nombre [tam1][tam2]...[tamN];
```

Ejemplo:

```
double tresd[5][10][4];
```

Tipos de Datos Estructurados.

Inicializado Arrays

Ejemplo: `double p[3]={1.0,2.0,3.0};`

Se puede omitir el tamaño inicial:

`double p[]={1.0,2.0,3.0};`

Ejemplo:

`int c[3][2]={{0,0},{1,1},{2,2}};`

Tipos de Datos Estructurados.

Inicializado Arrays

- Importante
 - Comienzo en 0 y hasta el tamaño del array-1
 - C no hace ninguna comprobación sobre el acceso a posiciones fuera de los límites del array

```
printf("%d",p[2]);  
printf("%d",c[2][1]);
```



Arrays unidimensionales

- Implementar un algoritmo que lea una serie de números, los almacene en un array y muestre la media de esos números.



Arrays bidimensionales

- Definir dos matrices de 3x3, realizar la suma y mostrar por pantalla.

Tipos de Datos Estructurados.

Registros o estructuras

```
struct strNombre{  
    Tipo1 miembro1;  
    Tipo2 miembro2;  
    ...  
};
```

- `strNombre` de la estructura
- Miembros de la estructura Cualquier tipo (simple, estructurado)
- Declaración posterior: `struct strNombre var;`

Tipos de Datos Estructurados. Ejemplo.

```
struct Punto{  
    int x, y;  
};  
struct Punto p1;  
printf("%d %d", p1.x, p1.y);
```

Definición de nuevos tipos.

```
struct Rectangulo{  
    struct Punto p1, p2;  
};
```

- Alternativamente:

```
typedef struct Punto strPunto;  
typedef struct Rectangulo{  
    strPunto p1,p2;  
} strRectangulo;
```

Tipos de Datos Estructurados.

Inicializando registros

```
struct Punto p1 = {2,3};  
struct Punto p2 = p1;  
  
printf("%d %d", p2.x, p2.y);
```

- Arrays de estructuras:

```
struct Punto p[10];  
p[0].x = 12; p[0].y = 14;
```

Tipos de Datos Estructurados

- Cálculo del centroide: El centroide de un conjunto de puntos es el promedio de todos los puntos. Si tienes n puntos $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, el centroide (x_c, y_c) se calcula como

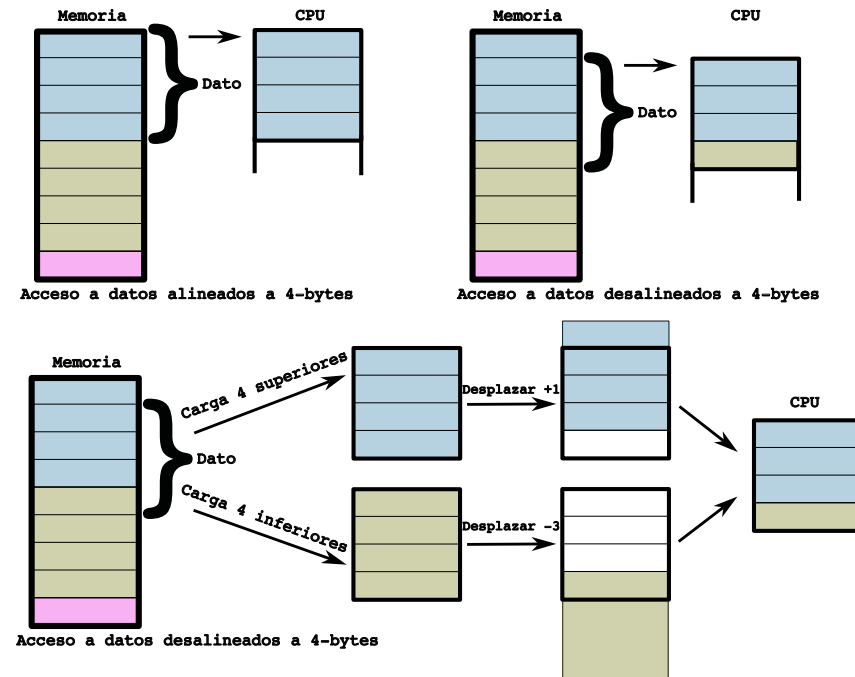
$$x_c = \frac{1}{n} \sum_{i=1}^n x_i$$

$$y_c = \frac{1}{n} \sum_{i=1}^n y_i$$



Datos Estructurados, almacenamiento

- La CPU no lee la memoria byte a byte. Lo hace en bloques para mejorar el rendimiento.
- Un dato debe alinearse a una dirección múltiplo de su propio tamaño, así se reducen las lecturas.



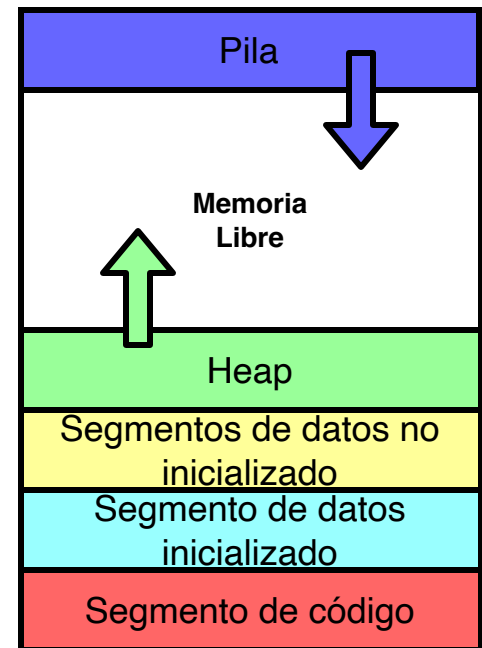


¿Ocupan lo mismo?

```
//Usa *sizeof*  
struct SinPadding  
{  
    char c;           // 1 byte  
    int i;            // 4 bytes  
    char c2;         // 1 byte  
};  
  
struct ConPadding  
{  
    char c;           // 1 byte  
    char c2;         // 1 byte  
    int i;            // 4 bytes  
};
```

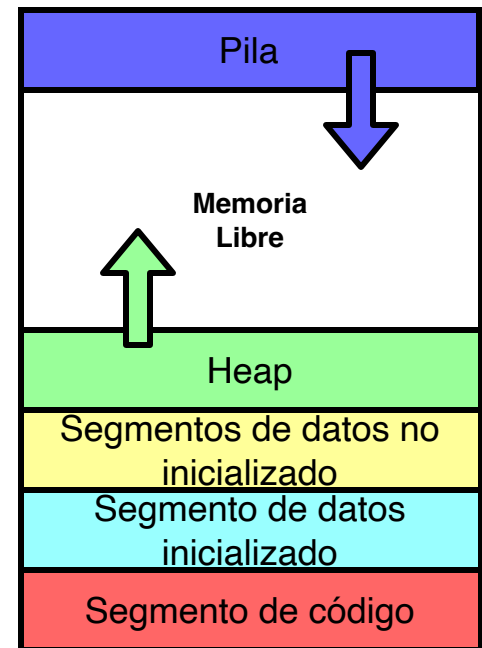
Estructura de memoria

- Pila (Stack)
 - Parámetros de función
 - Variables locales de función
 - Crece hacia abajo
- Montículo (Heap)
 - Memoria dinámica (malloc)
 - Crece hacia arriba



Estructura de memoria

- Segmento de Datos
 - Variables globales
- Segmento de Código
 - Código ejecutable del programa
 - Segmento de sólo lectura



Estructura de memoria

¿Puede crecer la memoria sin tope?

- **Segmentation fault** indica un error cuando accedes a una memoria que no existe o que no tienes permiso.

Reventando la memoria de C